# darkmpc, Milestone 2 walkthrough

The semi-honest three-party core

Tave La Rocque

4 November 2026

## Milestone 2: the semi-honest core

### What was built

The crate `darkmpc-mpc` implements three-party replicated secret sharing in Rust: arithmetic shares over $\mathbb{Z}_{2^{64}}$, Boolean shares of 64 bits at a time, multiplication and AND, opening, and secure comparison, together with minimum, maximum and selection built from it. Three parties run as three threads joined by in-memory channels. The protocol code is written exactly as it will run over the network later. Only the channel module changes in Milestone 3, when the parties become separate processes over TLS.

The security model of this crate is semi-honest with an honest majority: at most one party is corrupt, and it follows the protocol. Nothing here detects a party that sends wrong values. That is Milestone 4.

### Randomness without communication

Multiplication needs a fresh random sharing of zero every time (chapter 1 explains why). Sending that randomness would double the communication, so the parties generate it locally. At setup, party $i$ receives two 128-bit keys, $k_i$ and $k_{i+1}$, so each key is known to exactly two parties. Each key drives a stream of pseudorandom words, AES-128 applied to a counter: word $j$ of stream $k$ comes from $\mathrm{AES}_k(j)$. Both holders of a key use its stream for the same purposes in the same order, so they always read the same values without talking. For the zero-sharing, party $i$ takes

$$\alpha_i = F(k_i) - F(k_{i+1}),$$

and the three values cancel because every stream's word appears once with each sign.

AES comes from the RustCrypto `aes` crate (version 0.9.4). Its README states that it "has received one security audit by NCC Group, with no significant findings", funded by MobileCoin. It does not say which version was audited, so the write-up should say "an audited implementation" with that caveat rather than "audited code". The crate marks itself "hazmat" because a raw block cipher is not an encryption scheme. We

use it as a pseudorandom function, which is the building-block role it is meant for. Key setup in this milestone is done by the test harness. In deployment, each pair of servers agrees on its key over its TLS link.

## Multiplication and AND

Multiplication is the protocol of chapter 1, vectorised: a whole batch of products costs one round, and each party sends one 64-bit word per product. AND on Boolean shares is the same protocol with XOR in place of addition and AND in place of multiplication. Because a Boolean share is a 64-bit word, one AND operation processes 64 independent bits at once. This is called bit-slicing. It is also why the comparison circuit below is cheap to express: shifting a shared word left by $d$ positions is a local operation on each share, so "look $d$ bits down" costs nothing.

## Comparison

To decide whether $a < b$, compute $d = a - b$ (free), and take its most significant bit, which is 1 exactly when $d$ is negative as a signed 64-bit number. This is only correct if $a - b$ cannot wrap around $2^{64}$. The matcher's input bounds guarantee that every value it compares is below $2^{58}$ in absolute value. The difficulty is that $d$ is shared additively, $d = d_0 + d_1 + d_2$, and the top bit of a sum depends on every carry from below. The core computes those carries with a Boolean circuit in four steps.

**Step 1, local conversion.** Party $i$ holds $d_i$ and $d_{i+1}$ in the clear, so each summand $d_j$ has a trivial Boolean sharing in which one share is $d_j$ and the others are zero. No communication is needed.

**Step 2, three addends into two.** A carry-save adder replaces $d_0 + d_1 + d_2$ by $s + c$ with

$$s = d_0 \oplus d_1 \oplus d_2, \qquad c = \mathrm{maj}(d_0, d_1, d_2) \ll 1,$$

because at each bit position the three input bits sum to their XOR plus twice their majority. The majority costs one AND, using the identity

$$\mathrm{maj}(a, b, c) = \big((a \oplus b) \wedge (b \oplus c)\big) \oplus b.$$

If $a = b$, the AND is zero and the result is $b$, which is the majority. If $a \neq b$, the AND is $b \oplus c$, the result is $c$, and $c$ breaks the tie.

**Step 3, the carry into bit 63.** With $p = s \oplus c$ (propagate) and $g = s \wedge c$ (generate), the top bit of $s + c$ is $p_{63}$ XOR the carry into position 63. The carry is a prefix computation, done in the Kogge-Stone pattern: for $d = 1, 2, 4, 8, 16, 32$,

$$G \leftarrow G \oplus \big(P \wedge (G \ll d)\big), \qquad P \leftarrow P \wedge (P \ll d).$$

After the last step, bit $k$ of $G$ is the carry out of position $k$ taking all lower positions into account, so the answer is $p_{63} \oplus G_{62}$. The textbook rule combines with OR. XOR is

equivalent here because a position, or a block of positions, can never both generate a carry and propagate one, so the two terms are never 1 together. Each step is one round, and both ANDs in a step go in the same round.

**Step 4, back to arithmetic.** The comparison bit $b = b_0 \oplus b_1 \oplus b_2$ is Boolean-shared, but the matcher needs it as an arithmetic 0 or 1 to multiply with. Over the integers, $x \oplus y = x + y - 2xy$ for bits, so

$$e = b_0 + b_1 - 2\,b_0 b_1, \qquad b = e + b_2 - 2\,e\,b_2,$$

which costs two multiplications in sequence. With the bit in hand, $\min(a, b) = b + [a < b]\,(a - b)$ costs one more multiplication.

## Measured cost

A test pins these numbers, so any change to the protocol that changes the cost fails the build until the test is updated on purpose. Per element, per party:

| Operation | Words sent | Bytes sent | Rounds |
|---|---|---|---|
| Multiplication | 1 | 8 | 1 |
| Opening | 1 | 8 | 1 |
| Most significant bit | 13 | 104 | 8 |
| Comparison $a < b$ as 0/1 | 15 | 120 | 10 |

Rounds do not grow with batch size: a comparison of 100,000 pairs takes the same 10 rounds as one pair. That is the property the Stage 1 matcher is designed around. The comparison is not yet optimal. Kogge-Stone computes every prefix carry when only the top one is needed, and packing several comparisons into one word could cut the words sent. Neither change is worth making before the full matcher exists to measure against.

## How the tests are built

Eleven tests run the three parties on real threads and check every operation against plain arithmetic. That covers sharing and reconstruction (including detecting two disagreeing copies of a summand), linear operations (also checking that they send nothing), multiplication, chained multiplication, AND, the most significant bit, comparison, minimum and maximum, conversion of bits to arithmetic, the carry circuit on chosen inputs, comparison at its boundaries, and the communication costs above. Test values deliberately include 0, 1, the all-ones word, $2^{63}$, powers of two, and $2^k - 1$, since those are where wraparound and carry bugs live. Two further tests examine a single party's view for leakage.

Four bugs are planted behind feature flags, and `scripts/check_planted.sh` passes only if each one breaks the test suite.

**A wrong product share** (one cross term dropped) is caught by every multiplication test.

**A missing re-randomisation** is the important one. Every correctness test still passes with it, because the shares still add up to the right product. Only the leakage test catches it. In that test, party 2 multiplies secret zeros 4,096 times and computes, from its own shares and the message it received, the statistic

$$t = z_0 + x_0 y_2 + x_2 y_0 + x_0 y_0.$$

Chapter 1 showed that without the mask $t = x_0 y + y_0 x$, which is exactly 0 when both secrets are 0. With the mask, $t$ is uniformly random. A chi-squared test on the low byte of $t$ separates the two decisively. This is the clearest example in the project of a bug that correctness testing cannot see.

**An off-by-one in comparison** (computing $a \leq b$) is caught by the boundary test at $a = b$.

**A carry circuit one level short** (stopping at distance 16, so carries cannot travel more than 32 positions) taught a lesson I had wrong at first. I expected long carry chains to be rare, since the summands are random. They are not. The carry-save adder produces $s + c = d + w \cdot 2^{64}$ for some $w$. When $d$ is small and non-negative, $w$ is almost always positive, so $s + c$ wraps past $2^{64}$ while every result bit above $d$'s highest set bit is 0. The only way that happens is a carry running through all of those positions and out of the top: for a size-like value below $2^{40}$, a chain of more than 20 positions, and for values near zero, almost the whole word. Chain length is set by the secret value, not the shares, and dark pool values are small. So the short circuit fails on almost every comparison of small numbers. Several tests catch it, and a dedicated test drives the carry circuit with chosen inputs that force chains of every length from 1 to 63. The general point: a uniformly random 64-bit test value is a poor stand-in for the values a system actually sees.

## Next

Milestone 3 turns the reference mechanism into MPC on top of this core: oblivious grouping by hidden symbol, prefix sums, the clamp formula for fills, minimum-fill elimination for a fixed number of passes, and routing each fill back to its trader. It is differentially tested against the reference on random batches, and the parties move into separate processes with simulated network latency for the first benchmarks.